Welcome to your next lesson: Iterating over dictionaries: keys, values, and items in action.

I'm preparing your personalized lesson based on what you've learned, what's coming next, and the best way to help you practice.

Step by step, you’re moving forward.

Imagine you have a small price list in Python:

Python

```
prices = {
    "apple": 1.2,
    "banana": 0.5,
    "orange": 0.9
}
```

You already know how to loop over a list. With a dictionary, we can loop too — but now we can choose to walk over the labels (keys), the numbers (values), or both together. That’s what this lesson is about. 🔑

To start simple:

If I write:

Python

```
for item in prices:
    print(item)
```

this will loop over the keys: apple, banana, orange.

Make sense so far — that for item in prices: is really “loop over all the keys”?

Yes, that makes sense

Great 👍

Now I want to show you the clear way to say “loop over the keys”.

Using our same dictionary:

Python

```
prices = {
    "apple": 1.2,
    "banana": 0.5,
    "orange": 0.9
}
```

We can write:

Python

```
for item in prices.keys():
```

```
    print(item)
Step by step:


prices.keys() means: “give me all the keys (labels) from this dictionary”.
for item in ...: walks through those keys one by one.
print(item) shows each key: apple, banana, orange.

So for item in prices: and for item in prices.keys(): do the same thing, but the second one makes it obvious you’re looping over keys. 🔑

Does this step feel clear: prices.keys() = “all the keys from the dictionary”?

Yes, it's clear

Nice 😄


Next tiny idea: once you have a key from the loop, you can use it to get the value.


Using the same dictionary:

Python

prices = {
    "apple": 1.2,
    "banana": 0.5,
    "orange": 0.9
}

for item in prices.keys():
    print(item, prices[item])
Step by step for the first loop round:


item becomes "apple"
prices[item] means prices["apple"], which is 1.2
So it prints: apple 1.2

Then it does the same for banana and orange.

Does this “use the key to look up the value” pattern make sense?

Yes, it makes sense

Perfect 👍


Now let’s look at values only.


Same dictionary:

Python

prices = {
    "apple": 1.2,
    "banana": 0.5,
    "orange": 0.9
}

for price in prices.values():
    print(price)
Step by step:


prices.values() = “all the numbers (values) in the dictionary”
The loop goes through: 1.2, then 0.5, then 0.9
```

It does not give you the keys (apple, banana, orange), only the values.

So this pattern is for when you don’t care which key it came from, you just need all the values.

Does for price in prices.values(): = “loop over all the values” feel clear?

Yes, that's clear

Awesome 😊

You’ve seen:

Looping over keys: for item in prices.keys():
Looping over values: for price in prices.values():

Now let’s do keys and values together in one loop.

Python

```
prices = {
    "apple": 1.2,
    "banana": 0.5,
    "orange": 0.9
}

for item, price in prices.items():
    print(item, price)
```

Step by step for the first round:

prices.items() gives pairs: ("apple", 1.2), ("banana", 0.5), ("orange", 0.9)
item, price grabs each pair into two variables
So it prints:
apple 1.2
banana 0.5
orange 0.9

So .items() = key and value together in each loop.

Does this pattern with for item, price in prices.items(): make sense so far?

Yes, that's clear

Great, you’ve got the three patterns 😊

Now let’s connect them to when you’d use each one. Simple rule of thumb:

Use .keys() when you only care about the names (like product names, usernames, IDs).
Use .values() when you only care about the data (like prices, scores, ratings).
Use .items() when you need both name and data together (like printing full messages).

Quick example:

You want to check if any price is too high → loop over prices.values()
You want to print each item and its price → loop over prices.items()
You want to list all item names in a menu → loop over prices.keys()
Let’s do a tiny practice in your code editor 💻

Open the editor for this lesson.

Create a dictionary named movies with 3 entries like this (you choose the titles and ratings):

```
movies = {
    "Some Movie": 8.5,
    "Another Movie": 7.9,
    "Third Movie": 9.1
}
```
Under it, write a loop that prints only the movie titles, one per line, using .keys().

So the structure is:

Python

```
for title in movies.keys():
    print(title)
```
When you're done, hit Run to check it, then Submit code so I can review it.

Python

```
movies = {
    "Jaws": 8.5,
    "Jaws II": 7.2,
    "Jaws III": 5.6
}
for movie_name in movies.keys():
    print(movie_name)
```

Nice work, Ron! 🎬

Your code:

Python

```
movies = {
    "Jaws": 8.5,
    "Jaws II": 7.2,
    "Jaws III": 5.6
}
for movie_name in movies.keys():
    print(movie_name)
```
Output:

```
Jaws
Jaws II
Jaws III
```
This is exactly the right use of .keys(): you loop over the titles only and print them. ☑

Next small step: let's practice .items().

In the same file, under your existing code, add a new loop that prints lines like:

```
Jaws scored 8.5
Jaws II scored 7.2
Jaws III scored 5.6
```

Hint structure:

Python

```
for movie_name, rating in movies.items():
    print( ??? )
```
Use both movie_name and rating in the print.

Add that, hit Run to see the output, then Submit code so I can check it.

Python

```
movies = {
    "Jaws": 8.5,
    "Jaws II": 7.2,
    "Jaws III": 5.6
}
for movie_name in movies.keys():
    print(movie_name)

for movie_name, movie_rating in movies.items():
    print(f"{movie_name} scored {movie_rating}")
```

What do you think? Pick the option that best matches “I only care about the numbers (prices), not the names.” 👇

You are building a feature that checks whether any product price is over 100. You do NOT need product names, only the numbers. Which is the best way to loop over the dictionary? The dictionary looks like:

```
prices = {"laptop": 999, "mouse": 25, "keyboard": 75}
```

A.

```
for name in prices.keys():
```

B.

```
for item in prices.items():
```

C.

```
for price in prices.values():
```

D.

```
for thing in prices:
```

Nice job, Ron — that’s exactly the right choice 🎯

For this feature, you said: I only care about the numbers (prices), not the names.

`prices.values()` gives you just the values: 999, 25, 75
Looping like `for price in prices.values():` lets you directly check each number, e.g.:
Python

```
for price in prices.values():
    if price > 100:
        print("Too expensive")
```

No need to touch the keys at all — clean and simple.

You’ve now correctly used `.keys()`, `.items()`, and chosen `.values()` for the right scenario. 👏

Do you want one more quick practice choosing between keys / values / items, or are you feeling ready to wrap up this lesson?

One more practice, please

What do you think? Pick the option that best matches “I only need the student names.” 👇

You have a dictionary of student scores:

```
scores = {
    "Alice": 95,
    "Bob": 82,
    "Cara": 78
}
```

You want to print messages like:

Alice passed
Bob passed
Cara passed
You only need each student’s name once (you don’t care about the exact number score here, just their names).

Which is the best way to loop?

A.

```
for name in scores.keys():
```

B.

```
for score in scores.values():
```

C.

```
for name, score in scores.items():
```

D.

```
for thing in scores:
```

Good call asking for more practice — that’s how this really sticks 👍

In that question, the best loop was:

Python

```
for name in scores.keys():
    print(name, "passed")
```

Reason, step by step:

You said you only care about names (Alice, Bob, Cara).
scores.keys() gives exactly that: "Alice", "Bob", "Cara".
So for name in scores.keys(): is the simplest, clearest way.

Your choice:

Python

```
for name, score in scores.items():
    ...
```

also works, but it gives you both name and score. That’s extra data you don’t need for this feature, so we prefer .keys() here.

Let’s do one more quick scenario in words (no code yet):

You have:

Python

```
inventory = {
    "apple": 10,
    "banana": 3,
    "orange": 0
}
```

You want to print:

apple: 10 left
banana: 3 left
orange: 0 left

You need both the fruit name and the amount left.

In your own words: would you choose keys, values, or items for this?

how do you know if they pased if hnot read score

Great question 👌 and your thinking is right: in real life, to decide if they passed, you'd of course need to look at the score.

The scenario in that question was a bit simplified: it said "you don't care about the exact number, just their names," and it already assumed they passed. So it was more like: "We already know all these students passed; just print their names with the word 'passed'."

In real code where you must decide pass/fail, you'd use both name and score, like this:

Python

```
scores = {"Alice": 95, "Bob": 82, "Cara": 78}

for name, score in scores.items():
    if score >= 70:
        print(name, "passed")
```

Here .items() makes sense because you need both:

name → to print
score → to check if they passed
For the inventory example from my last message:

Python

```
inventory = {
    "apple": 10,
    "banana": 3,
    "orange": 0
}
```

You want:

apple: 10 left
banana: 3 left
orange: 0 left

You need both the fruit name and the amount.

Given what we just talked about, which would you pick here: keys, values, or items?

items

Exactly right — .items() is the best choice here ☑

For:

Python

```
inventory = {
    "apple": 10,
    "banana": 3,
    "orange": 0
}
```

You want to print:

apple: 10 left
banana: 3 left
orange: 0 left

You need both:

the key → "apple", "banana", "orange"
the value → 10, 3, 0

So this pattern fits perfectly:

Python

```
for fruit, amount in inventory.items():
    print(fruit, ":", amount, "left")
```

Here, each loop gives you a pair: (fruit, amount) — that's exactly what .items() is for. 🎯

You've now:

Used .keys() to loop over names/titles ☑
Used .values() when you only care about numbers ☑
Used .items() when you need both ☑
Chosen the right one from word problems ☑

You've hit the mastery goal for this lesson. 🎉

Ready to wrap up this lesson and move on to the next one?

Yes, let's move on

Awesome work today, Ron 🎉

Quick recap of what you now know how to do:

Loop over keys with .keys() when you care about the names/labels:

```
for name in scores.keys():
    print(name)
```

Loop over values with .values() when you only need the numbers/data:

```
for price in prices.values():
    print(price)
```

Loop over both key and value together with .items():

```
for item, price in prices.items():
    print(item, price)
```

Choose the right one based on the feature:
Names only → .keys()
Numbers only → .values()
Both together → .items()
You wrote solid code, asked a sharp question about "how do we know they passed", and reasoned your way to using .items() when both pieces are needed. That's exactly the kind of thinking that will make you strong as an engineer 💪

You're all set to hit Next lesson when you're ready. If anything about dictionaries feels fuzzy later, you can always ask me and we'll zoom back in on it together.

```
movies = {
    "Jaws": 8.5,
    "Jaws II": 7.2,
    "Jaws III": 5.6
}
for movie_name in movies.keys():
    print(movie_name)

for movie_name, movie_rating in movies.items():
```

```
530         print(f"{movie_name} scored {movie_rating}")
531
```